

Deep Learning for Beginners

★ Deep Learning

> It is a collection of statistical techniques of machine learning for learning feature hierarchies that are actually based on artificial neural networks.

● Deep Learning vs Machine Learning

features	ML	DL
Data	Performs well with a medium or small datasets.	Need a big dataset
Hardware requirements	works with low-end machines.	Requires machine with GPU.
Engineering peculiarities	Needs to understand the basic functionality of the data	Understand
Engineering peculiarities	Understands the features and how they represent the data.	Needs to understand the basic functionality of the data.
Training Time	very short	short long
Processing Time	A few seconds or hours	A few hours or weeks
No. of Algorithms	Many	few
Data Interpretation	Some ML Algorithms are easy to interpret whereas some are not.	Difficult.

★ Neurons and Neural Networks

→ A neuron in the context of a neural network is a small mathematical unit that receives inputs, processes them, and produces an output. It's inspired by biological neurons in the brain, but it's much simpler.

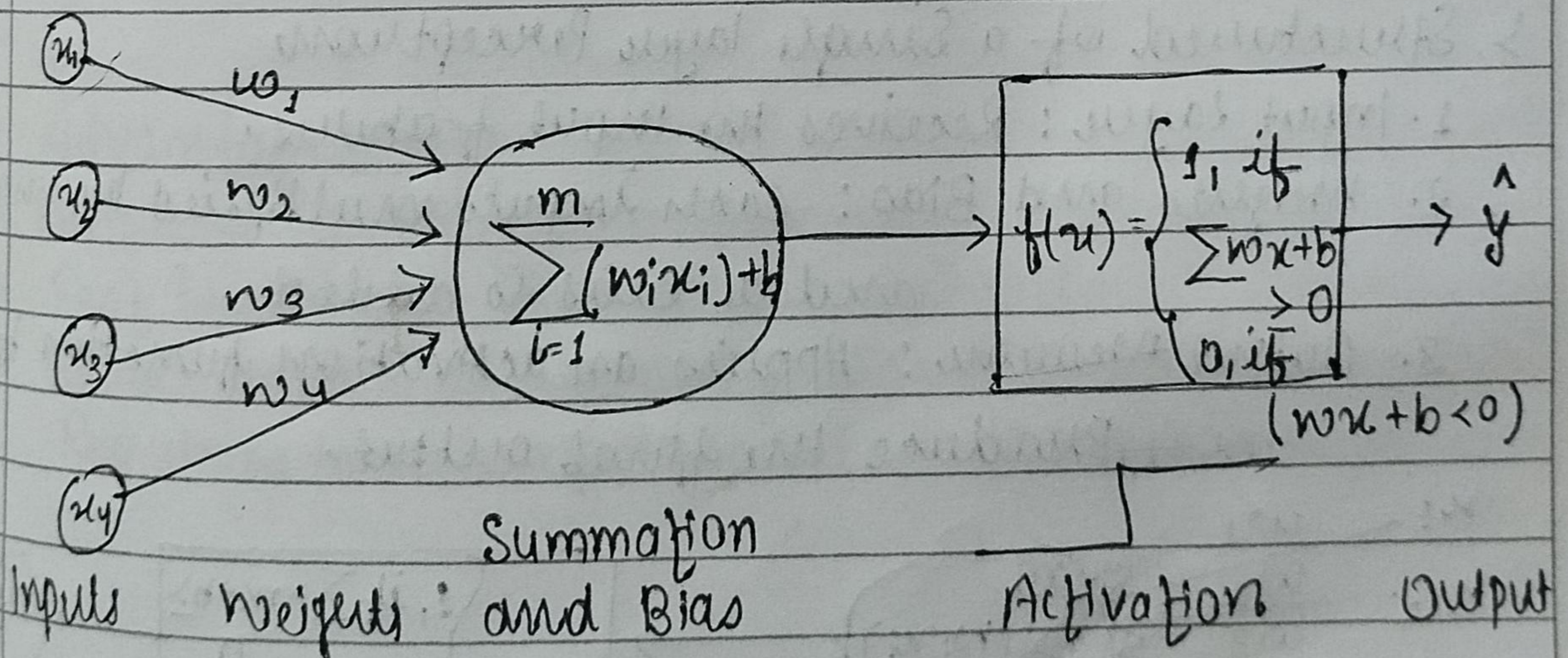
→ Receives inputs: It takes one more input values (e.g., x_1, x_2, x_3).

→ Applies weights: Each input is multiplied by a weight (w_1, w_2, w_3). These weights determine how important each input is.

→ Add a bias: A bias term (b) is added to help the neuron learn more flexible patterns.

→ Compute the weighted sum:

$$Z = w_1x_1 + w_2x_2 + w_3x_3 + b$$



★ Types of Deep Learning Networks

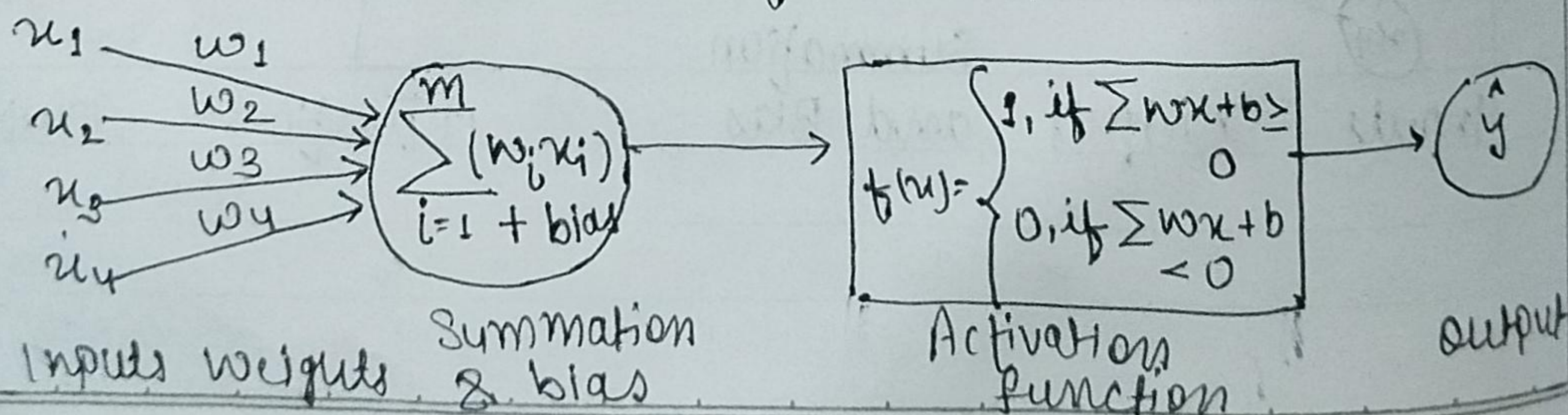
- > Perceptron (Single layer perceptron)
- > Feed Forward Networks
- > Multi layer Perceptron
- > Multi-layer Perceptron or Artificial Neural Networks
- > Radial based Networks
- > Convolutional Neural Networks
- > Recurrent Neural Networks
- > Long short-Term Memory Networks

● Single layer Perceptron

> A Single layer Perceptron (SLP) is the simplest type of artificial neural networks. It consists of one layer of trainable weights that connects the input directly to the output. It is mainly used for binary classification problems where the data is linearly separable.

> Structure of a Single layer Perceptron

1. Input layer: Receives the input features.
2. Weights and Bias: Each input multiplied by a weight and a bias is added.
3. Output Neuron: Applies an activation function to produce the final output.



• Working

1. Input values are fed into the perceptron
2. Each input is multiplied by its corresponding weight
3. The weighted inputs are summed along with the bias.
4. The activation functions determines the output in the form of (0 or 1).
5. During training, the weights are adjusted to reduce classification errors.

• Perceptron Learning Rule

$$w_{\text{new}} = w_{\text{old}} + \eta (t - y) x$$

whereas

$$w = \text{weight}$$

$$\eta = \text{learning rate}$$

$$t = \text{target output}$$

$$y = \text{Predicted output}$$

$$x = \text{Input}$$

• Advantages

- > Simple and Easy to implement
- > Fast training for simple datasets
- > Works well for linearly separable data.
- > Requires low computational resources

● Limitations

- ↳ Can solve only linearly separable problems.
- ↳ Cannot solve problems like XOR function.
- ↳ Produce only binary outputs
- ↳ Limited learning capacity compared to multilayer neural networks.

● Application

- ↳ Binary classification
- ↳ Spam email detection (simple cases)
- ↳ Pattern recognition
- ↳ Basic image classification
- ↳ Decision making system.

● Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from mtkind.plotting import plot_decision_regions
```

```
dataset = pd.read("Placement.csv")
dataset.head(3)
```

```
plt.figure(figsize=(4,3))
```

```
sns.scatterplot(x="CUPA", y="score", data=dataset, hue="placed")
```

```
plt.show()
```


Expt. No.

```
x = dataset.iloc[:, :-1]
```

```
y = dataset["placed"]
```

```
from sklearn.model_selection import train_test_split  
x_train, x_test, y_train, y_test = train_test_split(x, y,  
                                                    test_size=0.2, random_state  
                                                    = 42)
```

```
from sklearn.linear_model import Perceptron
```

```
pp = Perceptron()
```

```
pp.fit(x_train, y_train)
```

```
pp.score(x_train, y_train) * 100, pp.score(x_test, y_test)  
* 100
```

```
plot_decision_regions(x.to_numpy(), y.to_numpy(),  
                      clf=pp)
```

```
plt.show()
```


● Multi-layer Perceptron (Artificial Neural Network)

→ A multi-layer Perceptron (MLP) is a type of artificial neural network used in deep learning for solving classification and regression problems. It consists of multiple layers of interconnected neurons that learn complex patterns from data.

→ Structure of an MLP:

1. Input layer: Receives the input features.

Each neuron represents one input variable.

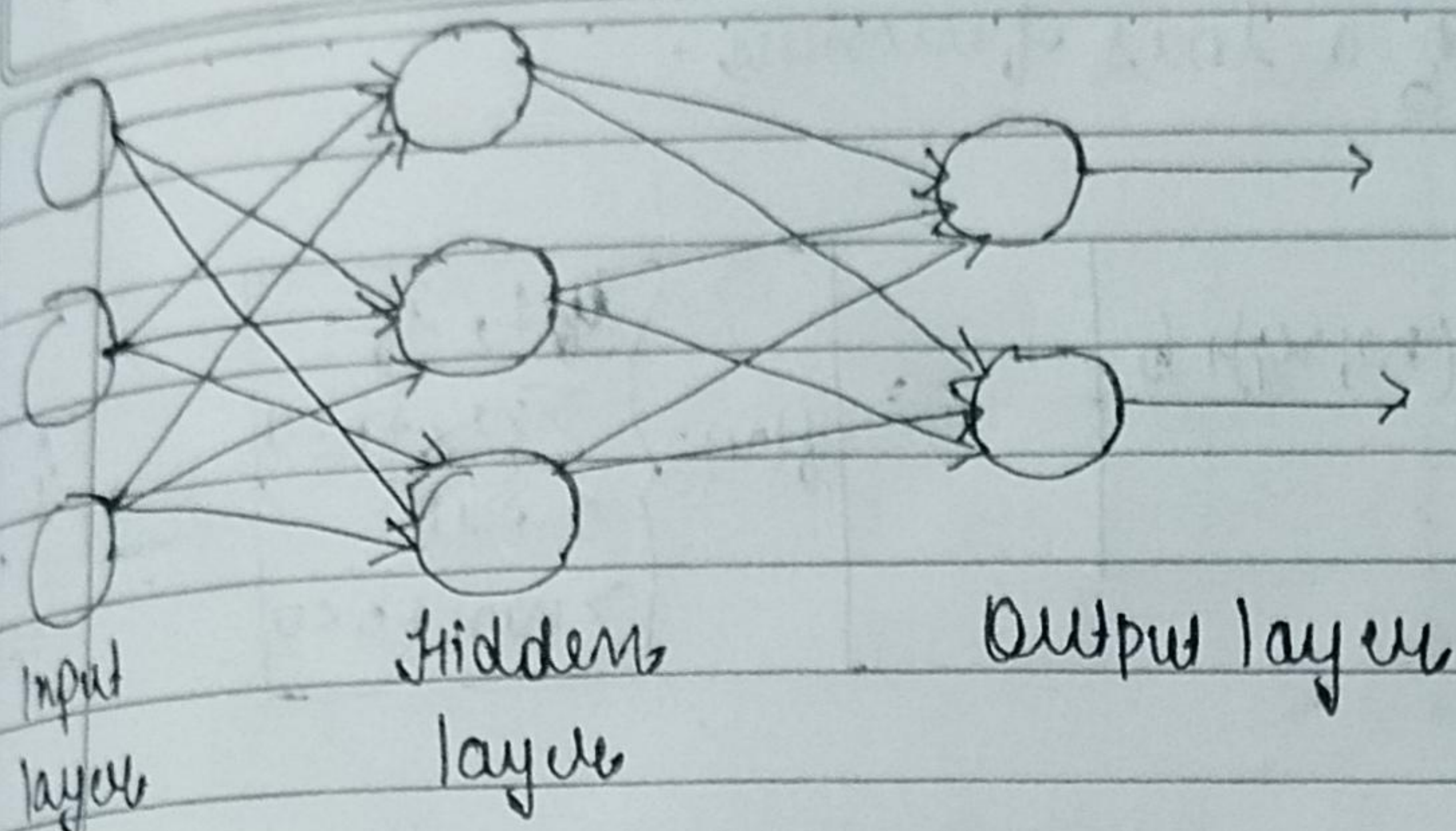
2. Hidden layer(s): One or more layers between the input and output.

Each neuron computes a weighted sum of inputs, adds a bias, and applies an activation function (such as ReLU, Sigmoid).

These layers enable the network to learn non-linear relationships.

3. Output layer: produces the final prediction.
The activation function depends on the task:
Sigmoid for binary classification
Softmax for multi-class classification
Linear for regression.

→ How an MLP works:



● Forward Propagation

> Forward propagation is the process of passing input data through the neural network to generate an output prediction.

> Steps :

1. Input data is fed into the input layer.
2. Each neuron calculates the weighted sum of inputs and add a bias.

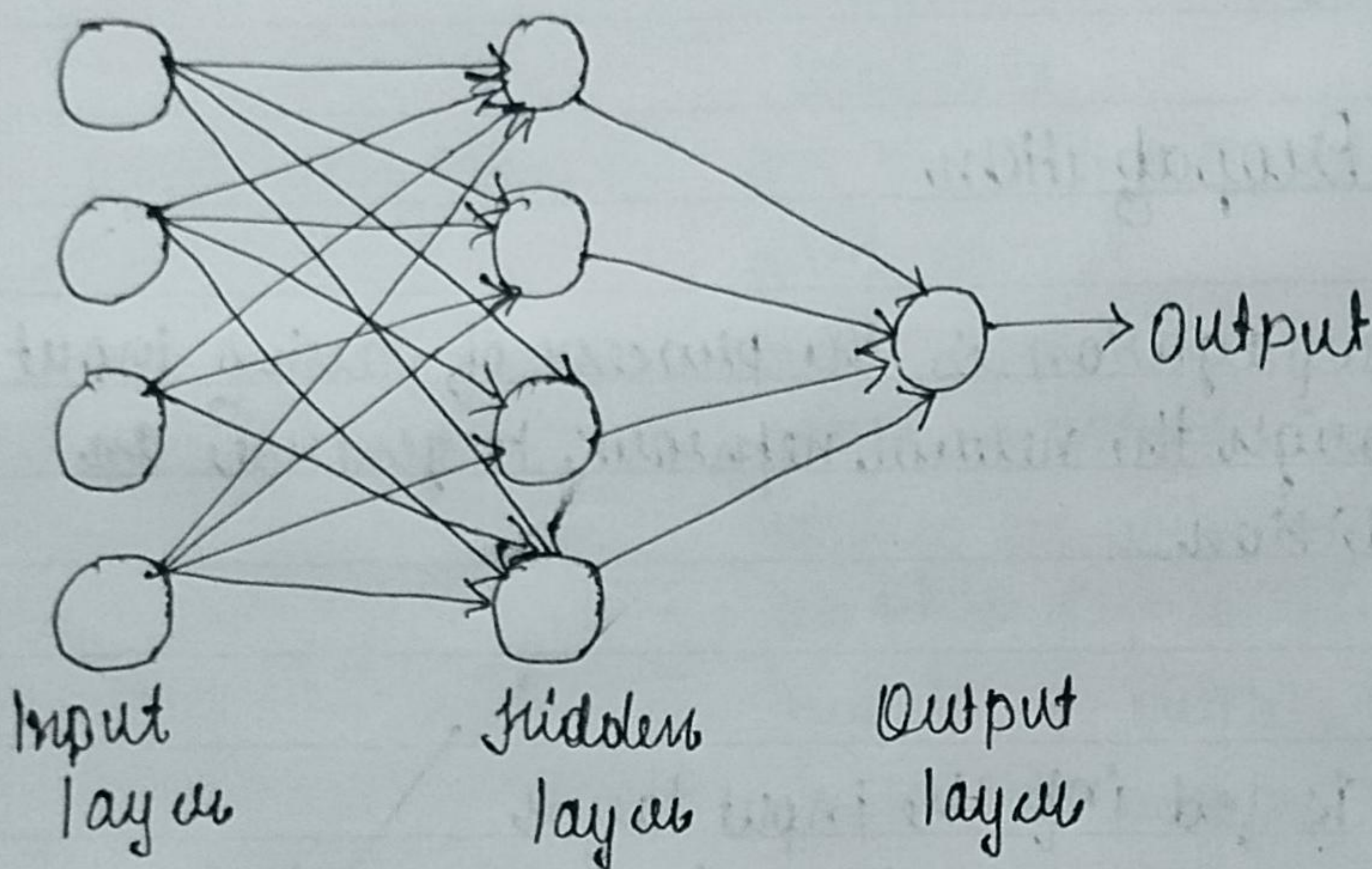
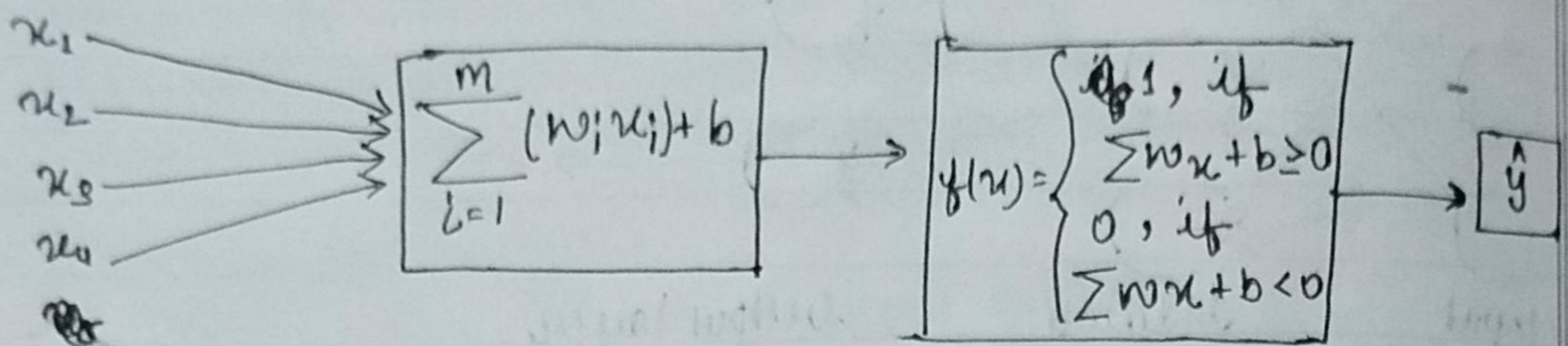
$$\hat{y} = \sum_{l=1}^m (w_l x_l) + \text{bias}$$

3. An activation function (ReLU, Sigmoid etc) is applied

$$f(x) = \begin{cases} 1, & \text{if } \sum w x + b \geq 0 \\ 0, & \text{if } \sum w x + b < 0 \end{cases}$$

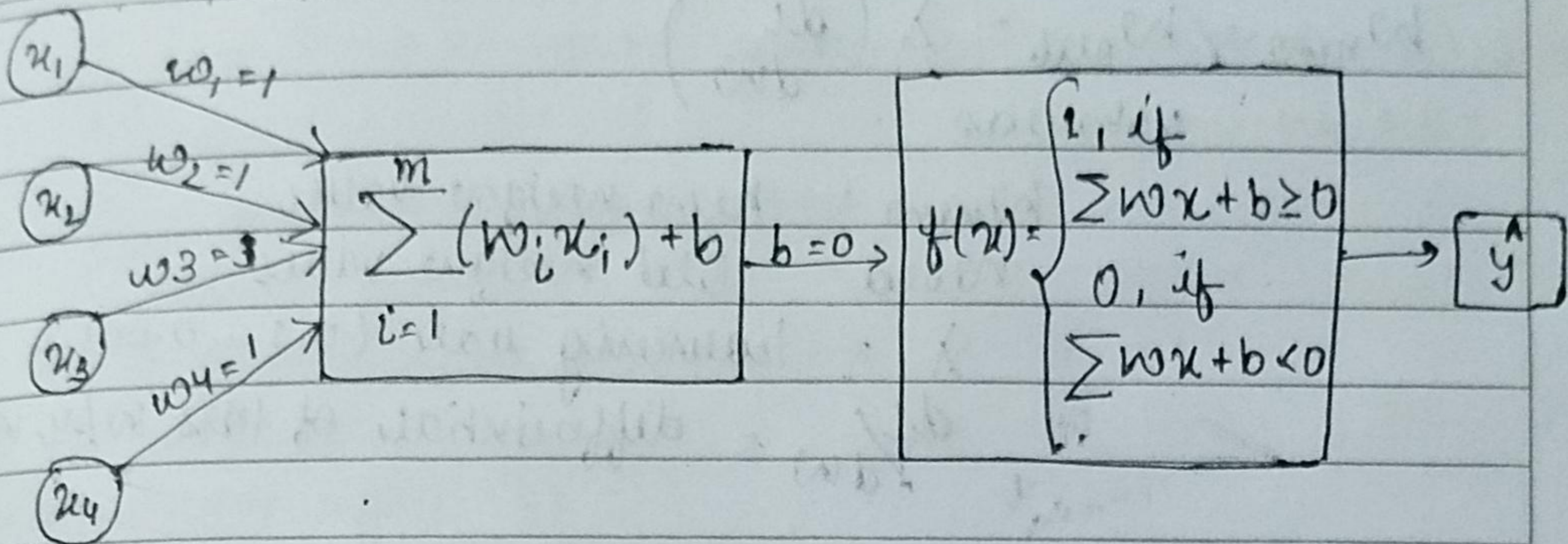
4. The output is passed through all hidden layers to the output layer.

5. The predicted output is compared with the actual output using a loss function.



★ Backward Propagation

➤ Backpropagation is one of the important concepts of a neural network. Our task is to classify our data best for this, we have to update the weights of parameters and bias.



Input layer Weight layer Summation & bias Activation function Output layer

$\hat{y}$	y_{original}
2	1
3	2
4	3
5	4

Steps of Backward Propagation:

1. Perform forward Propagation to compute the output
2. Calculate the Error (loss).

Error for a single output

$$\text{Error} = y_{\text{original}} - y_{\text{pred}} (\hat{y})$$

Or

Use Mean Squared Error

$$E = \frac{1}{2} (y_{\text{original}} - \hat{y})^2$$

3. Use Backward Propagation

gradient descent

$$w_{\text{new}} = w_{\text{old}} - \lambda \left(\frac{dL}{dw} \right)$$

whereas

w_{new} = new weight value

w_{old} = old weight value

λ = learning rate (0.1 - 0.001).

dL/dw = differential of loss w.r. weight

Imagine we have regression analysis,

use Mean Squared loss for single layer perceptron

$$L = \frac{1}{n} \sum (y - \hat{y})^2$$

$$L = \frac{1}{n} \sum (y - w_1 x_1 - w_2 x_2 - b)^2$$

$$\frac{dL}{dw_1} = \frac{1}{n} \sum (y - w_1 x_1 - w_2 x_2 - b)^2 (-x_1)$$

for Multilayer perceptron

$$w_{\text{new}} = w_{\text{old}} - \lambda \left(\frac{dL}{dw} \right)$$

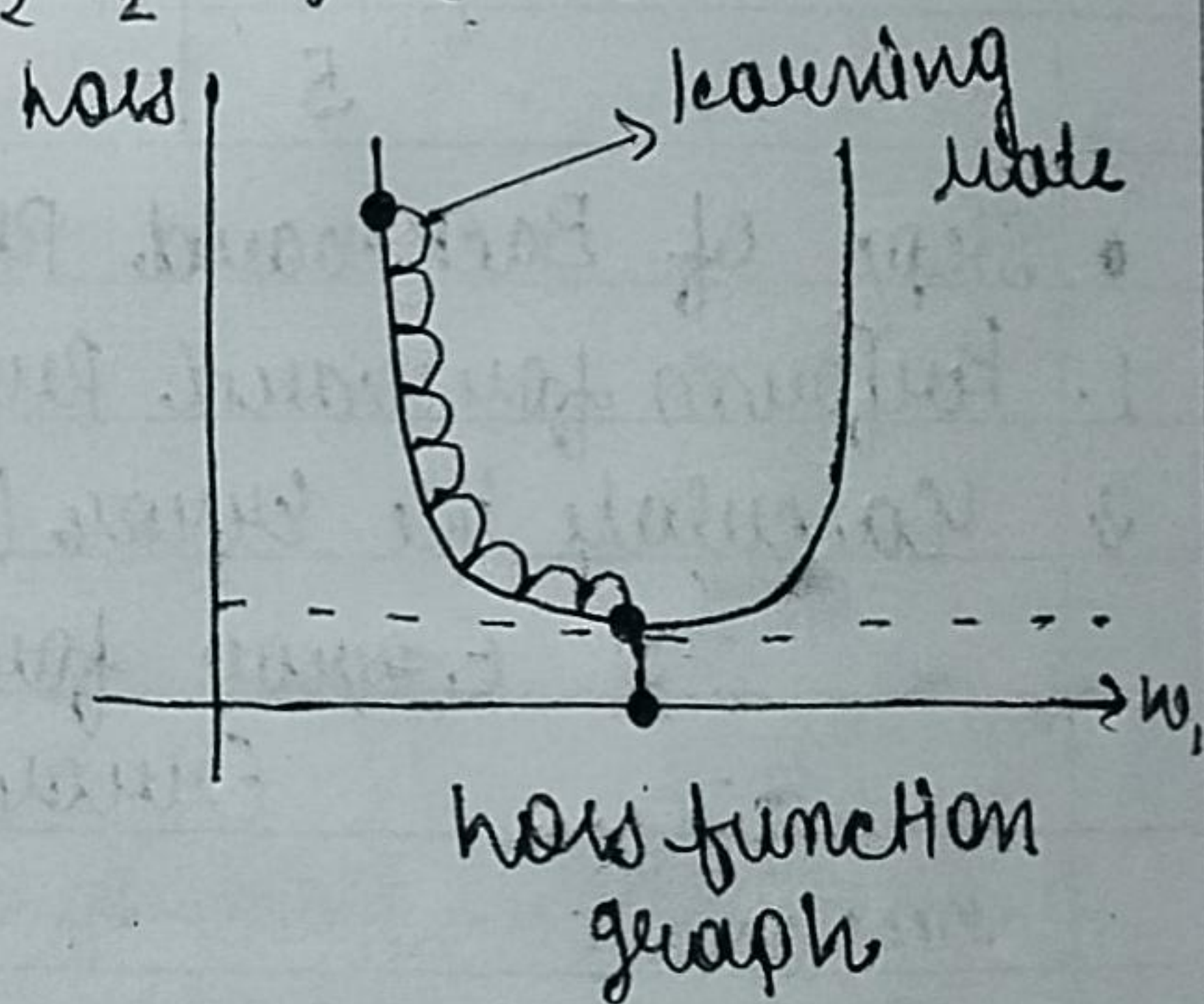
$$w_{5\text{new}} = w_{5\text{old}} - \lambda \left(\frac{dL}{dw_5} \right)$$

$$L = \frac{1}{n} \sum (y - \hat{y})^2$$

$$\hat{y} = o_1 w_5 + o_2 w_6 + b_3$$

$$\hat{y} = (x_1 w_1 + x_2 w_2 + b_1) w_5 + (\frac{x_1 w_3 +}{w_1 w_3 + o_1} x_2 w_4 + b_2) w_6 + b_3$$

$$\frac{dL}{dw_5} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dw_5}$$



$$\frac{dL}{dy} = \hat{y} - y = (x_1 w_1 + x_2 w_2 + b_1) w_5 + (x_1 w_3 + x_2 w_4 + b_2) w_6 + b_3$$

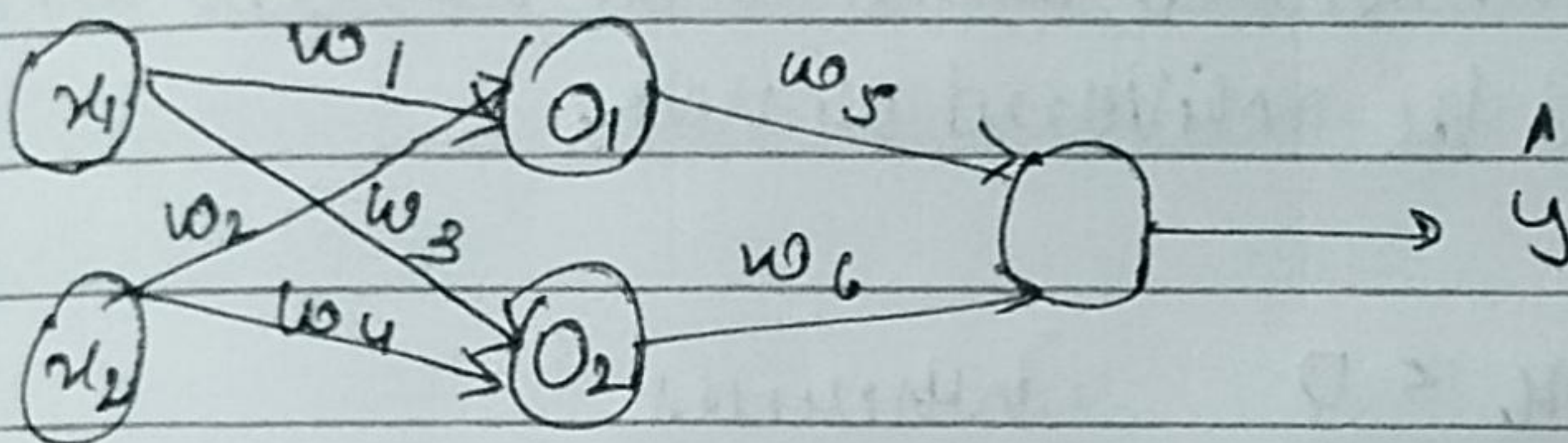
$$\frac{d\hat{y}}{dw_5} = x_1 w_1 + x_2 w_2 + b_1$$

notice that $x_1 w_1 + x_2 w_2 + b_1 = 0_1$

Final result will be

$$\frac{dL}{dw_5} = (\hat{y} - y) 0_1$$

4. Substitute in the last inside gradient descent for respective perceptron.



Multi layer perceptron.

Advantages of Multi layer Perceptron

1. Learn complex Patterns
2. Self-learning Adaptive
3. High accuracy
4. Handle noisy or incomplete data
5. Parallel Processing

★ Activation Function in Neural Networks

> An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction using simple mathematical operations.

> There are three types of Activation functions which are as follows:

- Binary Step Function
- Linear Activation Function
- Non-Linear Activation Function

• Binary Step Function:

> It depends on a threshold value that decides whether a neuron should be activated or not.

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases} \quad \text{whereas}$$
$$\hat{y} = x_1 w_1 + x_2 w_2 + b$$

• Linear Activation Function:

> The output of functions is not restricted in between any range. It's range is specified from $-\infty$ to ∞ .

$$y = x \quad \text{or} \quad f(x) = x$$

• Non-Linear Function:

> These are only one of the most widely used activation function. It helps the model in generalizing and adapting any sort of data in order to perform correct differentiation among the output.

There are four types of Non-Linear Neural Networks Activation Functions which are as follows:

- Sigmoid / Logistic Activation Function
- Tanh Function (Hyperbolic Tangent)
- ReLU Function
- Softmax Function

• Sigmoid / Logistic Activation Function

It's generally used to calculate binary function output which range moves from 0 to 1.

$$f(x) = \frac{1}{1 + e^{-x}}$$

• Tanh Function (Hyperbolic Tangent)

It's generally used to find RNN networks also it's differentiable and zero-centered ranged move from -1 to 1.

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

• Softmax Function:

Softmax converts a vector of values into probabilities whose sum is 1.

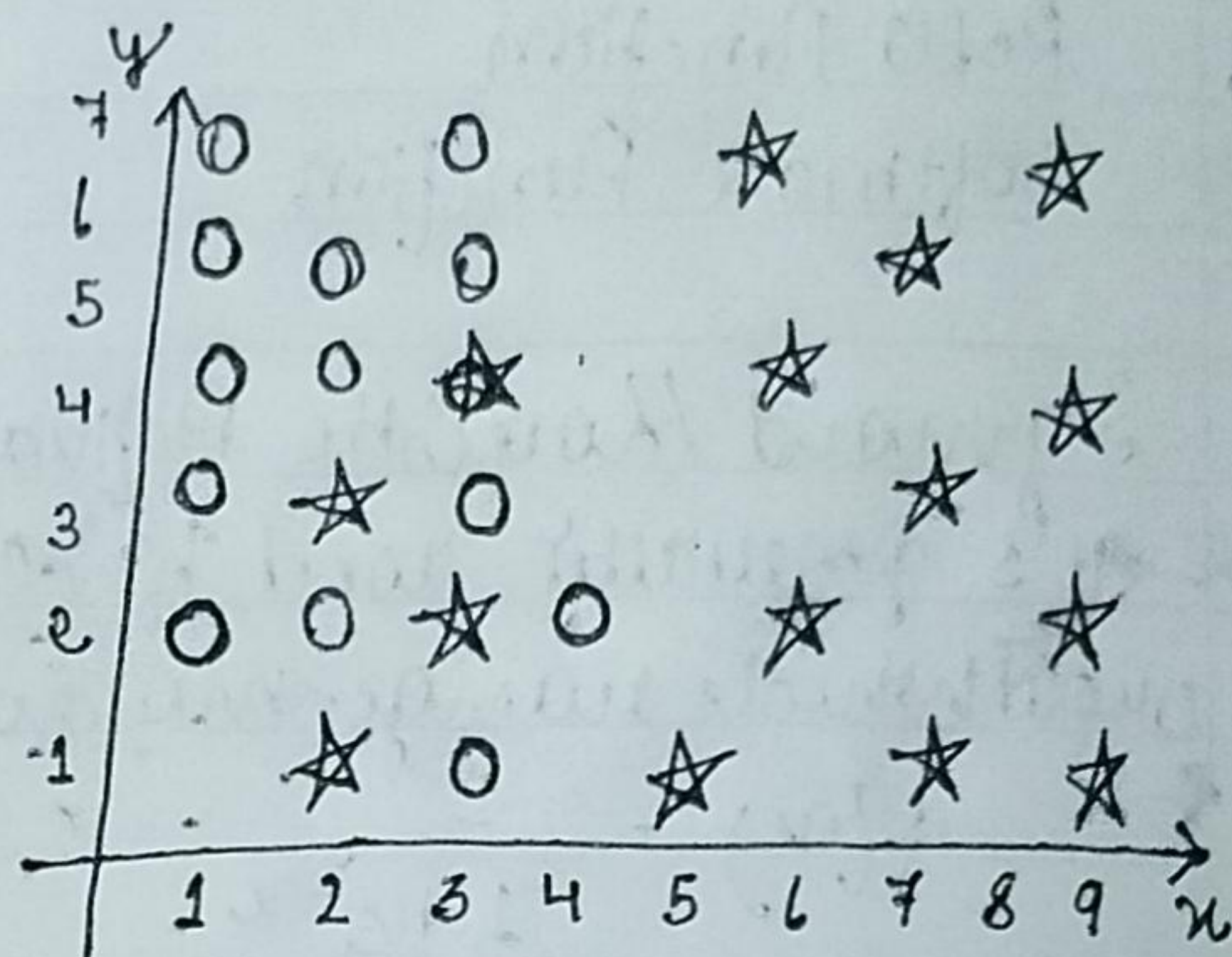
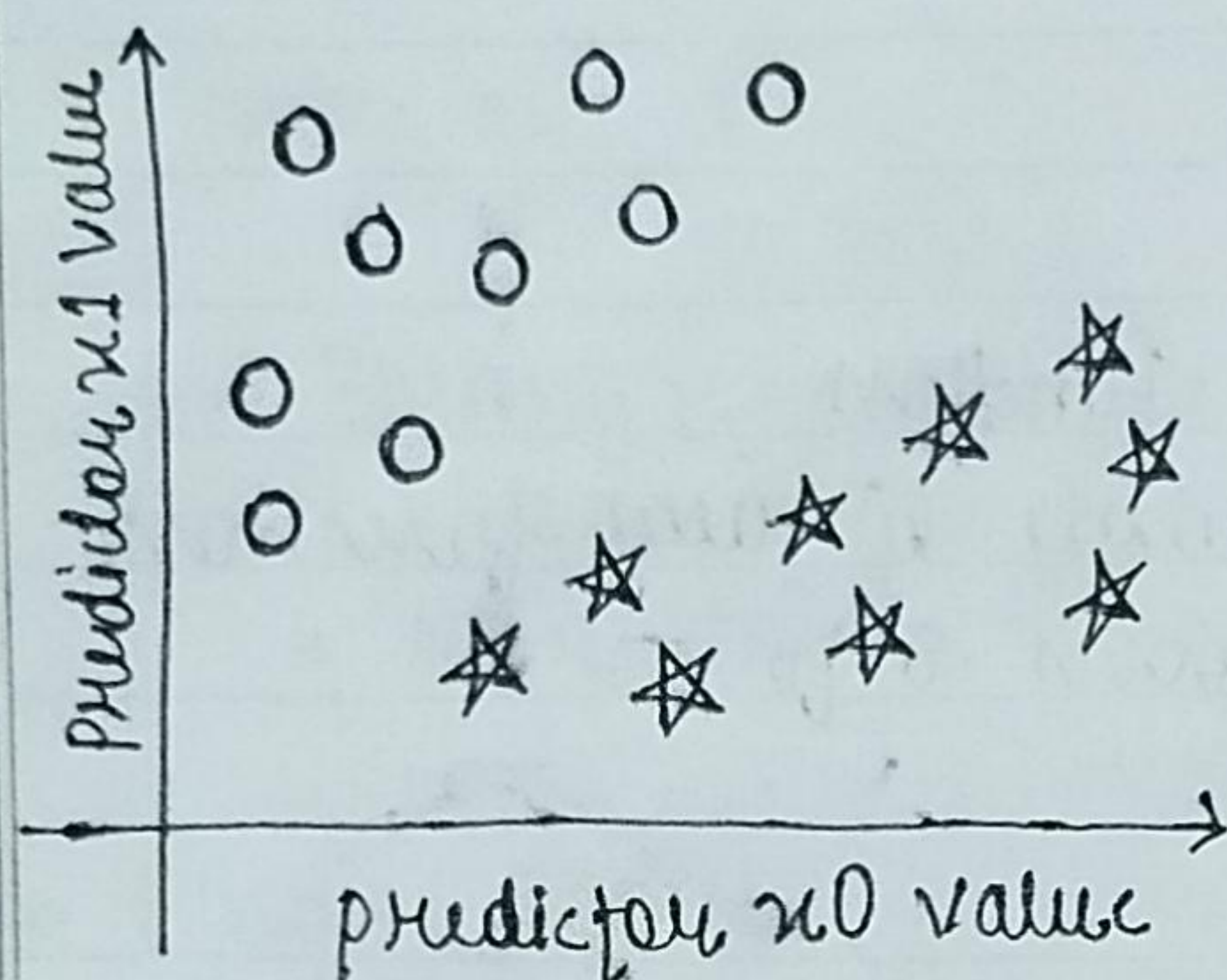
$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

• ReLU Function:

$$f(x) = \max(0, x)$$

★ Loss Function in Deep Learning

➤ The loss function is a method of evaluating how well your algorithm is modelling your datasets. It is a mathematical function of the parameters of the machine learning algorithm.



➤ There are three types of loss function are as follows:

1. Regression:

- MSE (Mean Squared Error)
- MAE (Mean Absolute Error)
- Huber loss

2. Classification:

- Binary cross-entropy
- Categorical cross-entropy

3. Auto Encoder:

- KL Divergence

4. GAN:

- Discriminator loss
- Minimax loss

Expt. No.

5. Object detection:

- Focal loss

6. Word embeddings:

- Triplet loss

1. Regression Loss:

- Mean Squared Error / Squared loss / L2 loss:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

It's differentiable loss that's mean we have to find Gradient descent =

$$w_{new} = w_{old} - \lambda \left(\frac{dL}{dw} \right)$$

- Mean Absolute Error / L1 loss:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

It's not differentiable actually that's mean we will not get exact predicted value but we will get approximate value. with Gradient descent:

$$\frac{dL}{dy} = \begin{cases} -1 & , \text{ if } y > \hat{y} \\ 1 & , \text{ if } y < \hat{y} \\ \text{undefined} & , \text{ if } y = \hat{y} \end{cases}$$

-1 if $y > \hat{y}$: prediction is smaller than actual value, so increase the prediction to reduce the error.

1 if $y < \hat{y}$: prediction is larger than the actual value, so decrease the prediction to reduce the error.

$y = \hat{y}$: error is zero, the derivative is undefined.

• Huber loss:

$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 \quad \text{if } |y_i - \hat{y}_i| \leq \delta$$

$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) \quad \text{if } |y_i - \hat{y}_i| > \delta$$

Whereas, n - the number of data points
 y - the actual value of the data point. Also known as true values
 $\hat{y}$ - the predicted value of the data point. This value is returned by the model.
 δ - defines the point where the Huber loss function transitions from a quadratic

> if your datasets have 70% prediction and 30% outliers then always use Huber loss.

2. Classification Loss

• Binary Cross Entropy / log loss:

$$\text{log loss} = -\frac{1}{N} \sum_{i=1}^N \left(y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i) \right)$$

> for output 0,

$$\text{log loss} = -\frac{1}{N} \sum_{i=1}^N \log (1 - \hat{y}_i)$$

> for output 1,

$$\text{log loss} = -\frac{1}{N} \sum_{i=1}^N (y_i \log \hat{y}_i)$$

> if your model prediction dataset output is in form of binary function such as 0 and 1 or cat or dog then always use Binary Cross Entropy.

• Categorical Cross Entropy:

$$Loss = - \sum_{j=1}^k y_j \log(\hat{y}_j)$$

where k is number of classes in data.

- if your model prediction dataset output is in form of categorical function such as cat, dog, cow etc then use Categorical Cross Entropy.

→ Take as an Example: cat, dog, cow

$$Loss = - \sum_{j=1}^k y_j \log(\hat{y}_j)$$

$$Loss = - (y_{cat} \log(\hat{y}_{cat}) + y_{dog} \log(\hat{y}_{dog}) + y_{cow} \log(\hat{y}_{cow}))$$

• Categorical Sparse Cross-Entropy:

- if target column has one hot encode to classes like 0 0 1, 0 1 0, 1 0 0 then use Categorical Cross Entropy.

- if the target column has numerical encoding to classes like 1, 2, 3, 4, ... n then use Sparse Categorical Cross-Entropy.

★ Optimization Techniques in Neural Networks

→ Optimizers are algorithms or methods used to change the attributes of your neural network such as weights and learning rate in order to reduce the losses.

→ There are various different types of optimizers as follows:

- Gradient Descent
- Stochastic Gradient Descent
- Stochastic Gradient Descent with momentum
- Mini-Batch Gradient Descent
- Adagrad
- RMSProp
- AdaDelta
- Adam

★ Customer Churn Prediction using ANN

• Code

```
import pandas as pd
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
```

```
dataset = pd.read_csv(r"C:\Users\Singer\Downloads\Churn-Modelling.csv")
```

```
dataset.head(3)
```

```
dataset.isnull().sum()
```

```
input_data = dataset.iloc[:, :-1]
```

```
output_data = dataset.iloc[:, -1]
```

```
ss = StandardScaler()
```

```
input_data = pd.DataFrame(ss.fit_transform(input_data),  
                           columns = input_data.columns)
```

```
input_data
```

```
input_data.shape
```

```
x_train, x_test, y_train, y_test = train_test_split(input_data,  
                                                    output_data, test_size=0.2, random_state=42)
```

```
x_train.shape
```


import keras

from keras.layers import Dense

from keras.models import Sequential

ann = Sequential()

ann.add(Dense(6, input_dim=8, activation='relu'))

ann.add(Dense(4, activation='relu'))

ann.add(Dense(2, activation='relu'))

ann.add(Dense(1, activation='sigmoid'))

ann.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

ann.fit(x_train, y_train, batch_size=100, epochs=50)

pred1 = ann.predict(x_test)

pred_data = []

for i in pred1:

if (i[0] > 0.5):

pred_data.append(1)

else:

pred_data.append(0)

pred_data

pred = ann.predict(x_train)

pred1_data = []

for i in pred:

if (i[0] > 0.5):

pred1_data.append(1)

else:

pred1_data.append(0)

pred1_data

from sklearn.metrics import accuracy_score
 accuracy_score(y_test, pred_data) * 100
 accuracy_score(y_train, pred_data) * 100

★ Improve The Performance

➤ Hyperparameters

- No of hidden layers
- No of node
- Activation Functions
- Loss Functions
- Optimizers
- Batch size
- No of Epochs
- Learning rate

➤ Vanishing / Exploding gradient descent

- Use optimizers
- Update initialise weights
- Batch normalization

➤ Data

➤ Slow training

➤ Overfitting

- L1 & L2 Regularization
- Early stopping
- Batch normalization

★ Identify Overfitting in Deep Learning

- Overfitting is a common explanation for the poor performance of a predictive model.
- Overfitting refers to an unwanted behaviour of a machine learning algorithm used in predictive modelling.
- Identify Overfitting: if train data > test data
 - Cross-validation
 - Train with more data
 - Remove features
 - Early stopping
 - Regularization
 - Ensembling
 - Hyperparameters

● Code (Early Stopping & Regularization)

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

dataset = pd.read_csv(r"C:\Users\Singh\Downloads\Chem-Modelling.csv")

dataset.head(3)
dataset.isnull().sum()
input_data = dataset.iloc[:, :-1]
output_data = dataset.iloc[:, -1]
ss = StandardScaler()
input_data = pd.DataFrame(ss.fit_transform(input_data),
                           columns = input_data.columns)
input_data
```



```
input_data.shape
```

```
x_train, x_test, y_train, y_test = train_test_split(input_data,  
                                                    output_data, test_size = 0.2, random_state  
                                                    = 42)
```

```
x_train.shape
```

```
import tensorflow
```

```
from keras.layers import Dense
```

```
from keras.regularizers import L2
```

```
from keras.models import Sequential
```

```
from keras.callbacks import EarlyStopping
```

```
ann = Sequential()
```

```
ann.add(Dense(6, input_dim=8, activation="relu",  
              kernel_regularizer=L2(12=0.01)))
```

```
ann.add(Dense(4, activation="relu", kernel_regularizer=  
              L2(12=0.01)))
```

```
ann.add(Dense(2, activation="relu", kernel_regularizer=  
              L2(12=0.01)))
```

```
ann.add(Dense(1, activation="sigmoid"))
```

```
ann.compile(optimizer="adam", loss="binary_crossentropy",  
            metrics=['accuracy'])
```

```
ann.fit(x_train, y_train, batch_size=100, epochs=50,  
        validation_data=(x_test, y_test), callbacks=  
        EarlyStopping())
```



```
train_accuracy = ann.history.history["accuracy"]  
test test_accuracy = ann.history.history["val_accuracy"]
```

```
plt.plot([i for i in range(1, 20)], train_accuracy)  
plt.plot([i for i in range(1, 20)], test_accuracy, c='red')  
plt.show()
```

```
pred = ann.predict(x_train)
```

```
preds_data = []
```

```
for i in pred:
```

```
    if (i[0] > 0.5):
```

```
        preds_data.append(1)
```

```
    else:
```

```
        preds_data.append(0)
```

```
preds_data
```

```
pred1 = ann.predict(x_test)
```

```
pred_data = []
```

```
for i in pred1:
```

```
    if (i[0] > 0.5):
```

```
        pred_data.append(1)
```

```
    else:
```

```
        pred_data.append(0)
```

```
pred_data
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(y_test, pred_data) * 100
```

```
accuracy_score(y_train, preds_data) * 100
```


★ Batch Normalization

Batch normalization (Batchnorm) is a technique used in Artificial Neural Networks (ANNs) to normalize the inputs of each layer during training. It helps the network train faster, improves stability and can reduce overfitting.

How Batch Normalization Works?

for each mini-batch:

1. compute the mean (μ) of the batch

2. compute the variance (σ^2) of the batch

3. Normalize each input: $\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$

where ϵ is a small constant to avoid division by zero.

4. Apply learnable scaling and shifting:

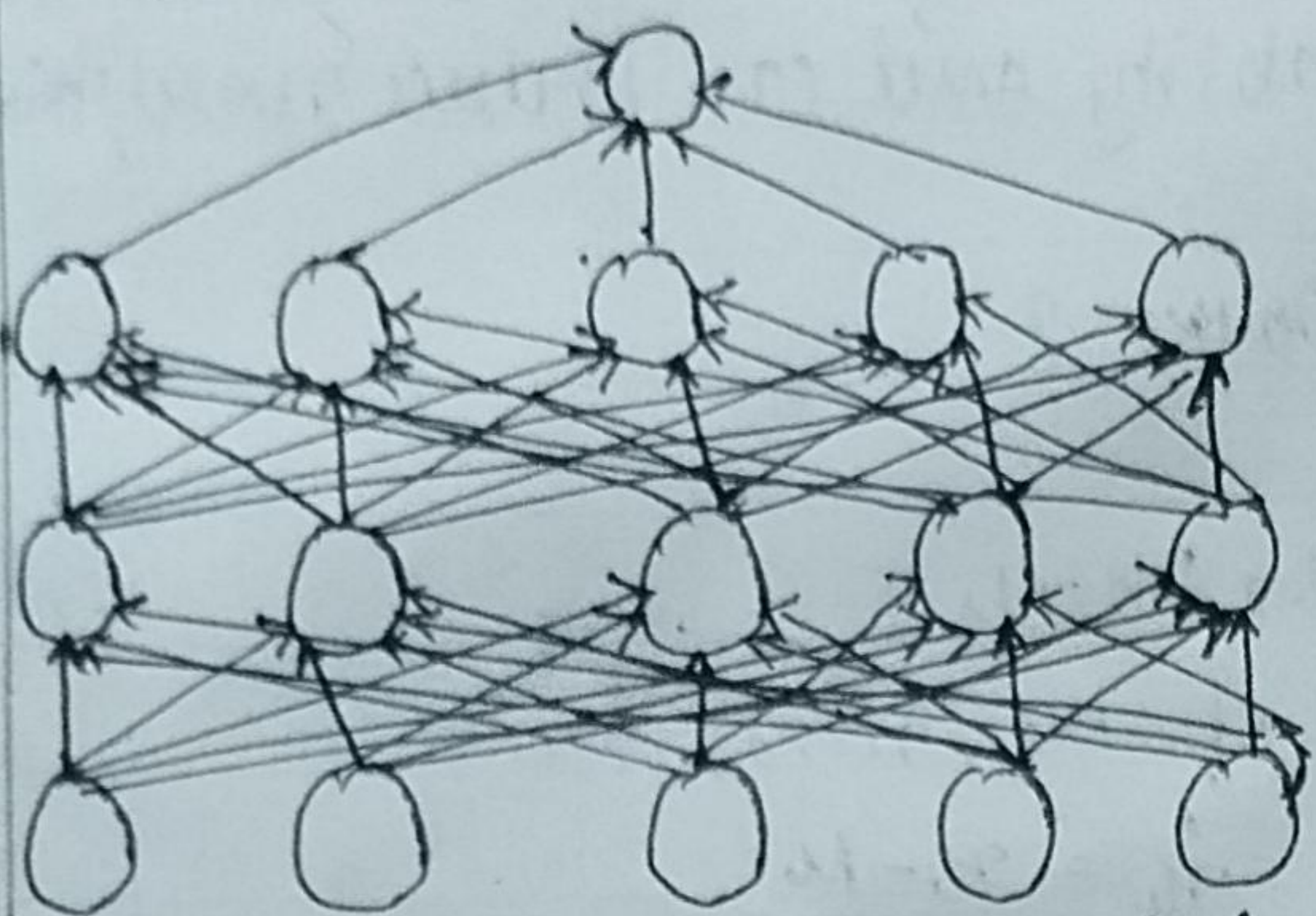
$$y = \gamma \hat{x} + \beta$$

where γ = learnable scale parameters

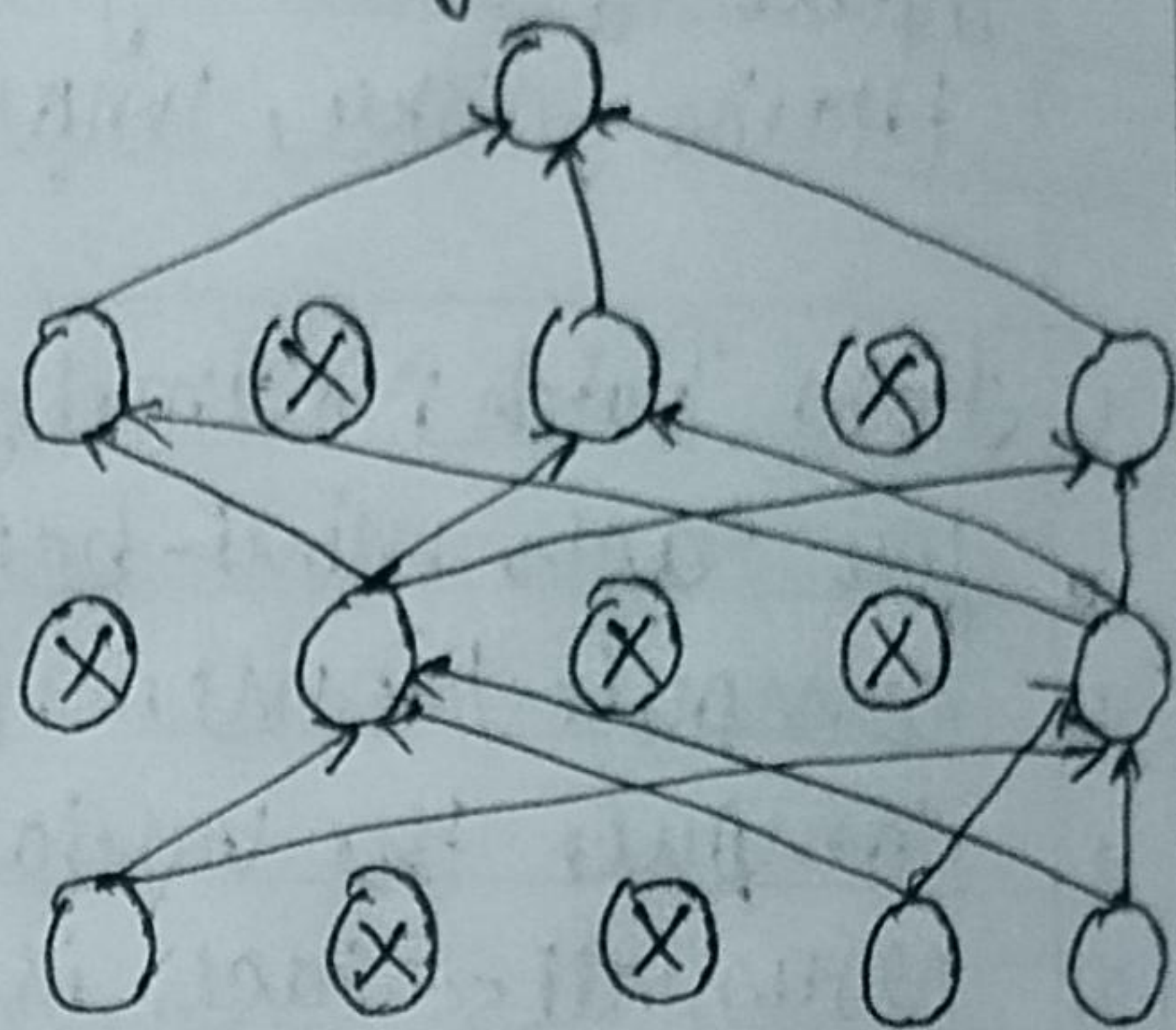
β = learnable shift parameters

★ Dropout Layer

> All the forward and backwards connections with a dropped node are temporarily removed, thus creating a new network architecture out of the parent network. The nodes are dropped by a dropout probability of p .



(a) Standard Neural Net



(b) After applying dropout

● Code

```
import pandas as pd
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
```

```
import matplotlib.pyplot as plt
```

```
dataset = pd.read_csv("Churn_Modelling.csv")
```

```
dataset.head(3)
```

```
input_data = dataset.iloc[:, :-1]
```

```
output_data = dataset.iloc[:, -1]
```

```
ss = StandardScaler()
```

```
input_data = pd.DataFrame(ss.fit_transform(input_data), columns=  
                           = input_data.columns)
```

```
input_data.shape
```


Expt. No.

```
x_train, x_test, y_train, y_test = train_test_split(input_data,
                                                    output_data, test_size = 0.2, random_state = 42)
```

```
x_train.shape
```

```
import tensorflow
```

```
from keras.layers import Dense, BatchNormalization, Dropout
```

```
from keras.regularizers import L2
```

```
from keras.callbacks import EarlyStopping
```

```
from keras.models import Sequential
```

```
ann = Sequential()
```

```
ann.add(Dense(6, input_dim = 8, activation = "relu", kernel-
        regularizer = L2(l2 = 0.01)))
```

```
ann.add(Dropout(0.5))
```

```
ann.add(Dense(4, activation = "relu", kernel_regularizer = L2(l2 = 0.01)))
```

```
ann.add(Dropout(0.5))
```

```
ann.add(Dense(2, activation = "relu", kernel_regularizer = L2(l2 = 0.01)))
```

```
ann.add(Dropout(0.5))
```

```
ann.add(Dense(1, activation = "sigmoid"))
```

```
ann.compile(optimizer = "adam", loss = "binary_crossentropy",
        metrics = ['accuracy'])
```

```
ann.fit(x_train, y_train, batch_size = 100, epochs = 50, validation-
        data = (x_test, y_test), callbacks = [EarlyStopping])
```

```
train_accuracy = ann.history.history['accuracy']
```

```
test_accuracy = ann.history.history['val-accuracy']
```



```

plt.plot([2 for i in range(1, 20)], train-accuracy)
plt.plot([1 for i in range(1, 20)], test-accuracy, c="red")
plt.show()

pred = ann.predict(x_train)
pred_data = []
for i in pred:
    if (i[0] > 0.5):
        pred_data.append(i)
    else:
        pred_data.append(0)

```

pred_data

```

preds = ann.predict(x_test)
pred_data = []
for i in preds:
    if (i[0] > 0.5):
        pred_data.append(i)
    else:
        pred_data.append(0)

```

pred_data

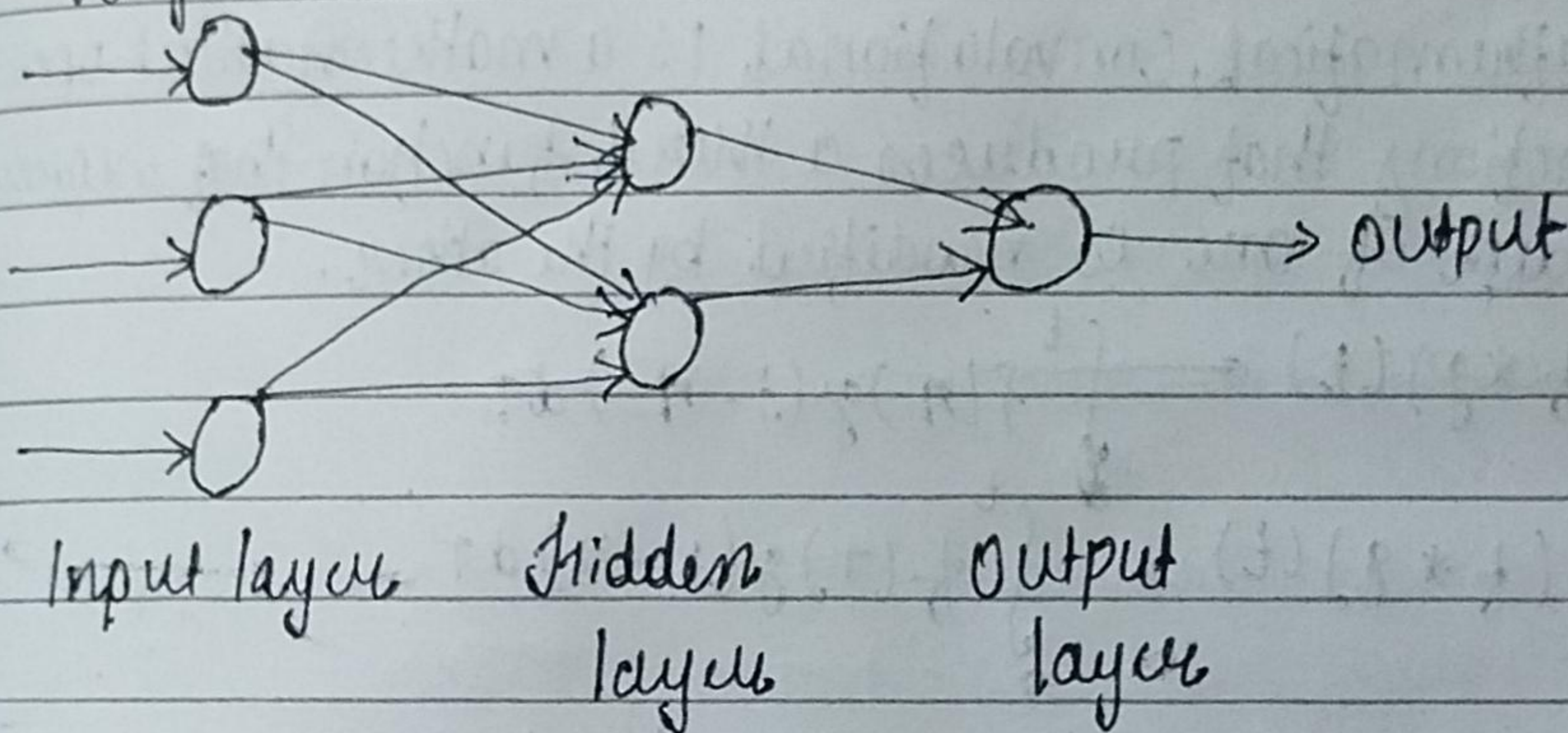
```

from sklearn.metrics import accuracy_score
accuracy_score(y_test, pred_data) * 100
accuracy_score(y_train, pred_data) * 100

```


★ Vanishing Gradient Problem

- > The Vanishing gradient Problem is encountered when training artificial neural networks with gradient-based learning methods and backpropagation. In such methods, during each iteration of training each of the neural network's weights receives an update proportional to the partial derivative of the error function with respect to the current weight.

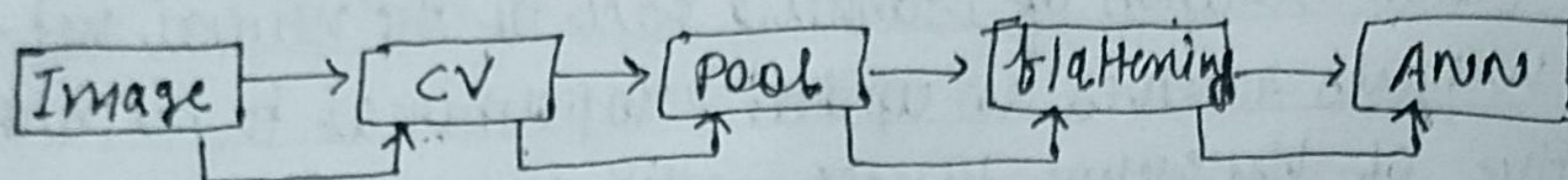


- > Why we will get to see Vanishing Gradient Problem
- Due to Deep neural networks $\rightarrow$ no. of hidden layers increases cause vanishing gradient
 - Due to Activation functions $\rightarrow$ Sigmoid and tanh function cause vanishing gradient problem

★ Convolutional Neural Network (CNN)

→ CNN is a type of artificial neural network, which is widely used for image/object recognition and classification. Deep learning thus recognizes objects in an image by using a CNN.

● How does it work naturally?



● Convolutional

Now in mathematical, convolutional is a mathematical operation on two functions that produces a third function that expresses how the shape of one is modified by the other.

~~$$c = (f * g)(t) = \int_0^t f(\tau)g(t-\tau)d\tau$$~~

$$c = (f * g)(t) = \int_0^t f(\tau)g(t-\tau)d\tau$$

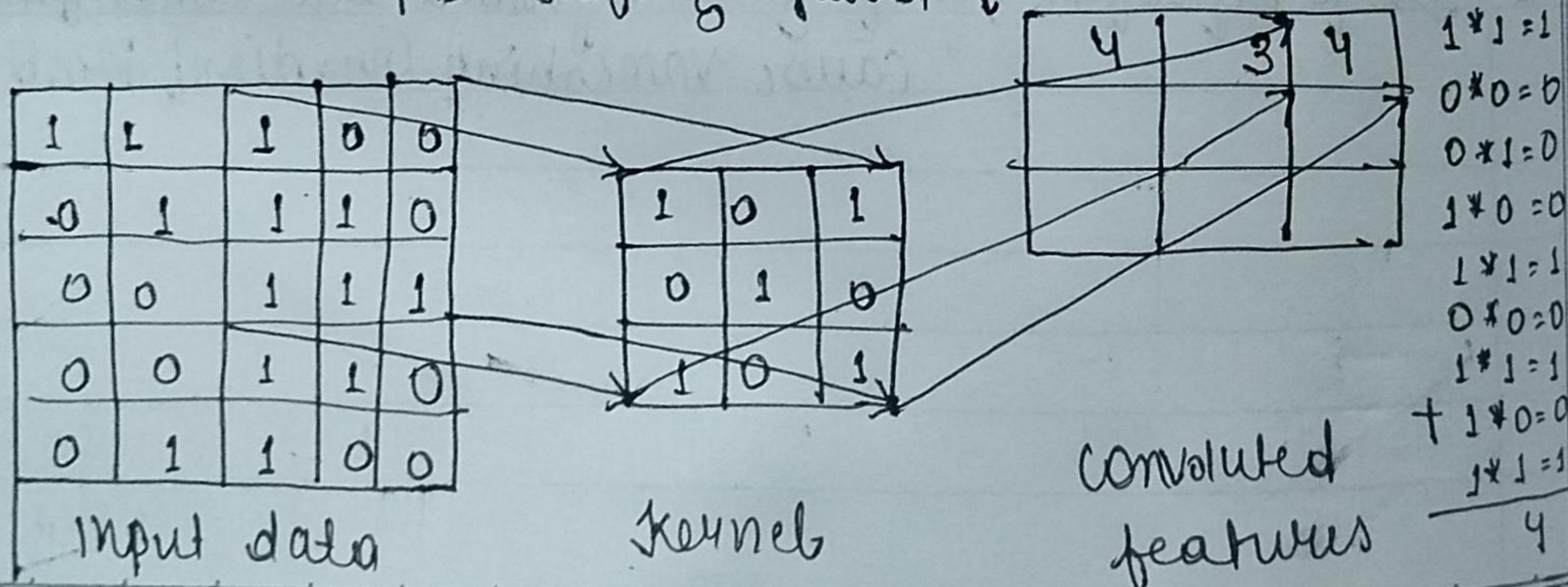
Whereas, c = convolution output function

f = original input function

g = function that is shifted over the input function

t = variable representing range of shifting

τ = shifting against t



Pooling

- A Pooling layer is another building block of a CNN. It's function is to progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network.

0	1	0	0	0
0	1	1	1	0
1	0	1	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

2x2

Max Pooling

1	1	0
4	2	1
0	2	1

Pooled Feature Map

Flattening

- Flattening is converting the data into a 1-dimensional array for inputting it to the next layer.
- Flatten output of the convolutional layers to create a single long feature vector
- It is connected to the final classification model, which is called a fully-connected layer.

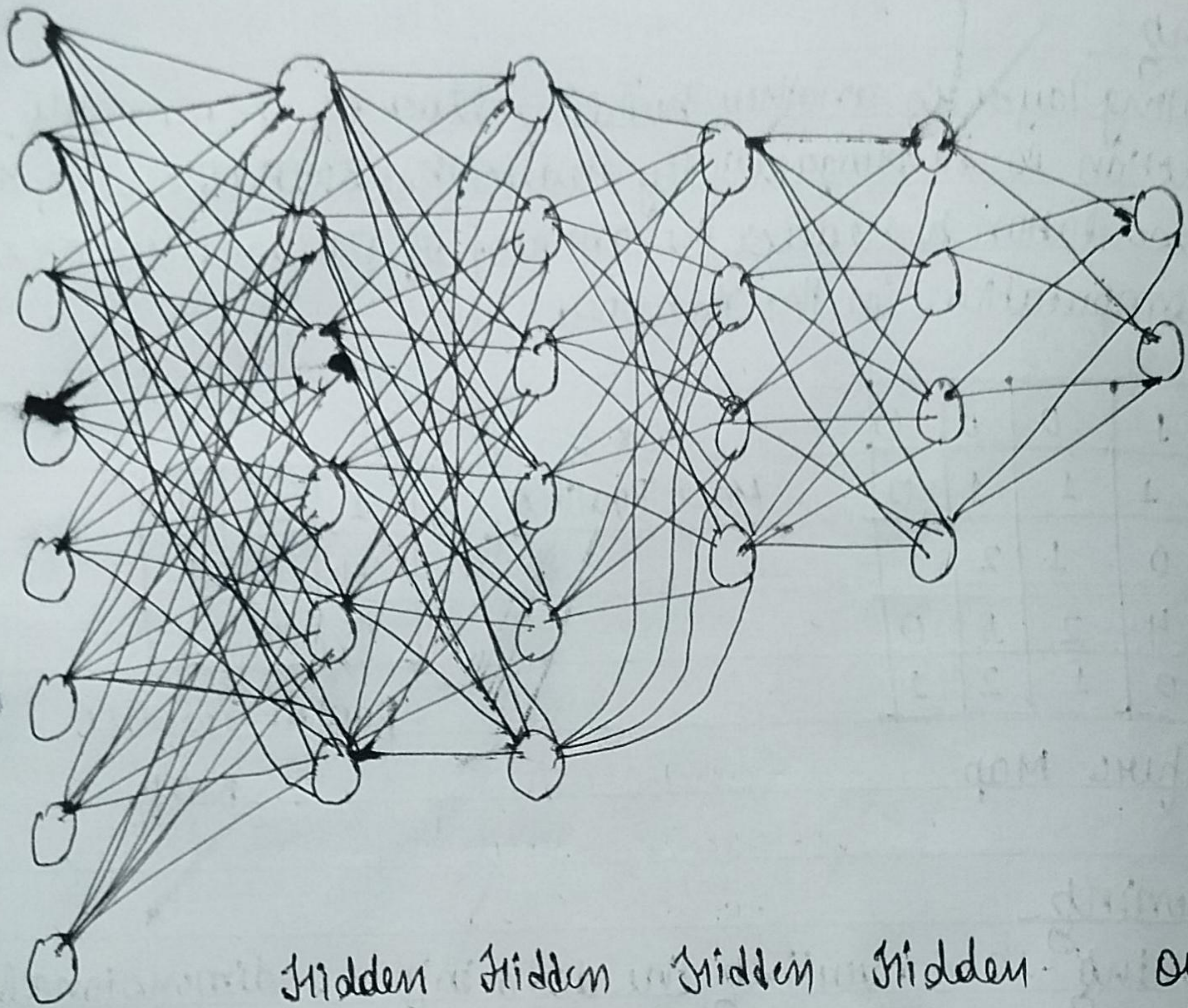
1	1	0
4	2	1
0	2	1

Pooled feature map

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
1	1	0	4	2	1	0	2	1

 $\Rightarrow \hat{y}$ output

• Fulling Connection (Anno)



input layer Hidden layer Hidden layer Hidden layer output layer

Code of Convolutional Neural Networks

import tensorflow

from keras.layers import Dense, Conv2D, MaxPooling2D, Flatten

from keras.models import Sequential

cnn = Sequential()

cnn.add(Conv2D(32, (3, 3), input_shape=(64, 64, 3), activation='relu'))

cnn.add(MaxPooling2D(pool_size=(2, 2)))

cnn.add(Conv2D(16, (3, 3), activation='relu'))

cnn.add(MaxPooling2D(pool_size=(2, 2)))

cnn.add(Flatten())

cnn.add(Dense(64, activation='relu'))

cnn.add(Dense(32, activation='relu'))

cnn.add(Dense(16, activation='relu'))

cnn.add(Dense(8, activation='relu'))

cnn.add(Dense(4, activation='relu'))

cnn.add(Dense(2, activation='relu'))

cnn.add(Dense(1, activation='sigmoid'))

cnn.compile(loss='binary_crossentropy', optimizer='adam')

from keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(

rescale = 1./255,

shear_range = 0.2,

zoom_range = 0.2

horizontal_flip = True)


```
test_data_gen = ImageDataGenerator(rescale = 1./255)
```

```
train_generator = train_data_gen.flow_from_directory(
```

```
    'data/train',
```

```
    target_size = (150, 150),
```

```
    batch_size = 32,
```

```
    class_mode = 'binary')
```

```
test_generator = test_data_gen.flow_from_directory(
```

```
    'data/validation',
```

```
    target_size = (150, 150),
```

```
    batch_size = 32,
```

```
    class_mode = 'binary')
```

```
model.fit_generator(
```

```
    train_generator,
```

```
    steps_per_epoch = 2000, nb_epoch = 50,
```

```
    nb_epoch = 50,
```

```
    validation_data = test_generator,
```

```
    nb_val_samples = 500)
```

```
from keras.preprocessing import image
```

```
img = image.load_img('any image', target_size = (64, 64))
```

```
img = image.img_to_array(img)
```

```
import numpy as np
```

```
img = np.expand_dims(img, axis=0)
```

```
nn.predict(img)
```

```
if P[0][0] < 0.5:
```

```
    print("Dog")
```

```
else:
```

```
    print("cat")
```


PyTorch Tutorial for Beginners

Phase → 0

● Settings.ipynb

```
import torch
```

```
x = torch.tensor([1, 2, 3, 4, 5])
```

```
x
```

```
print("hello")
```

```
import numpy as np
```

```
print(np.__version__)
```

```
import torch
```

```
print(torch.__version__)
```

```
import torch
```

```
x = torch.tensor([[1, 2], [3, 4]])
```

```
x
```

```
z = torch.zeros(2, 3)
```

```
z
```

```
o = torch.ones(2, 3)
```

```
o
```

```
h = torch.randn(2, 3)
```

```
h
```

```
s = torch.tensor([[1, 2, 3], [4, 5, 6]])
```

```
s.shape
```



```
i_id = touch.tensor([10, 20, 80, 40, 50])
```

```
print(i_id[0])
```

```
print(i_id[1])
```

```
i_2d = touch.tensor([11, 22, 33], [44, 55, 66])
```

```
i_2d[1][2]
```

```
a = touch.tensor([3, 6, 9])
```

```
b = touch.tensor([5, 10, 15])
```

```
print(a + b)
```

```
print(a * b)
```

Import numpy as np

```
arr = np.array([1, 2, 3])
```

```
n = touch.from_numpy(arr)
```

```
print(n)
```

● Linear ^{regression} ~~Polynomial~~ Prediction Model

import torch

```
x = torch.tensor([1., 2., 3., 4.])
```

```
y = torch.tensor([2., 4., 6., 8.])
```

```
w = torch.tensor(0.0)
```

```
b = torch.tensor(0.0)
```

```
y_pred = w * x + b
```

```
loss = ((y_pred - y) ** 2).mean()
```

```
print("predicted values", y_pred)
```

```
print("Actual values", y)
```

```
print("loss", loss)
```


Phase - 1

import torch

x = torch.tensor([1., 2., 3., 4.])

y_true = torch.tensor([2., 4., 6., 8.])

lr = 0.1

epochs = 10

for epoch in range(epochs):

 y_pred = w * x

 loss = ((y_pred - y_true) ** 2).mean()

 loss.backward()

 with torch.no_grad():

 w -= lr * w.grad

 print(f"Epoch {epoch+1}: w = {w.item():.4f},

 loss = {loss.item():.4f}")

• Sample code

~~Phase 2~~

x = torch.tensor(2.0)

w = torch.tensor(3.0, requires_grad=True)

y_pred = w * x

y_true = torch.tensor(10.0)

loss = (y_pred - y_true) ** 2

print(loss)

loss.backward()

print(w.grad)

print("Prediction", y_pred.item())

print("loss", loss.item())

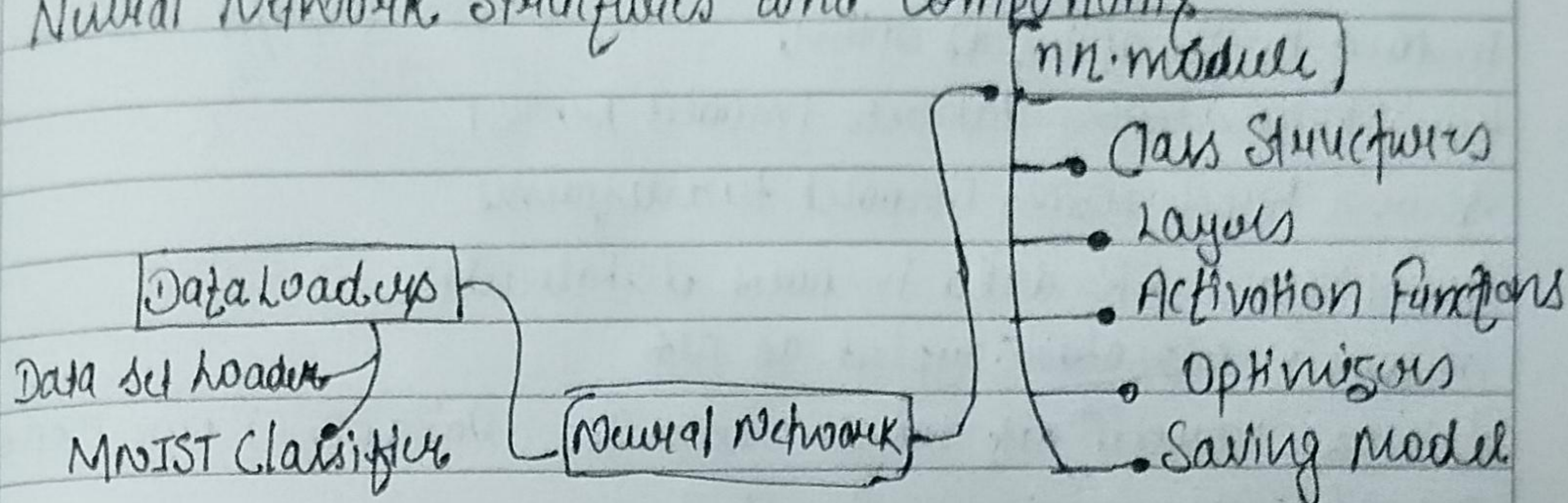
print("w.grad", w.grad.item())

• Sample Code-II

```
x = torch.tensor(2.0)
w = torch.tensor(3.0, requires_grad=True)
y_true = torch.tensor(10.0)
lr = 0.1
y_pred = w * x
loss = (y_pred - y_true) ** 2
loss.backward()
print("Before Update")
print("w:", w.item())
print("loss", loss.item())
print("gradient", w.grad.item())
with torch.no_grad():
    w -= lr * w.grad
w.grad.zero_()
print("After Update")
print("w:", w.item())
```


Phase - II

• Neural Network Structures and Components



• Code Sample (Optimisers)

```

import torch
import torch.nn as nn
import torch.optim as optim

class MyModel(nn.Module):
    def __init__(self):
        super.__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)

x = torch.tensor([1.], [2.], [3.], [4.])
y = torch.tensor([2.], [4.], [6.], [8.])
model = MyModel()
loss_fn = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.1)
for epoch in range(10):
    optimizer.zero_grad()
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    loss.backward()
    optimizer.step()
  
```

Print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

● MNIST Project Code

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision.datasets import MNIST
from torchvision import transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
device = "cuda" if torch.cuda.is_available() else "cpu"
print("using device ", device)
transform = transforms.ToTensor()
train_data = MNIST(
    root = "data",
    train = True,
    download = True,
    transform = transform
)
test_data = MNIST(
    root = "data",
    train = False,
    download = True,
    transform = transform
)
train_loader = DataLoader(
    train_data,
    batch_size = 64,
    shuffle = True
)
```



```
test_loader = DataLoader(
    test_data,
    batch_size = 64,
    shuffle = False
```

```
)
class DigitModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(28 * 28, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )
```

```
def forward(self, x):
    return self.net(x)
```

```
model = DigitModel().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr = 0.001)
epochs = 5
```

```
for epoch in range(epochs):
```

```
    model.train()
```

```
    total_loss = 0
```

```
    for image, labels in train_loader:
```

```
        images = images.view(images.size(0), -1).
            to(device)
```



```

labels = labels.to(device)
optimizer.zero_grad()
outputs = model(images)
loss = loss_fn(outputs, labels)
loss.backward()
optimizer.step()
total_loss += loss.item()
print(f"Epoch {epoch+1}/{epochs} | loss: {total_loss:.4f}")

```

```
model.eval()
```

```
correct = 0
```

```
total = 0
```

```
with torch.no_grad():
```

```
for images, labels in test_loader:
```

```
images images = images.view(images.size(0), -1).to(device)
```

```
labels = labels.to(device)
```

```
outputs = model(images)
```

```
predictions = output.argmax(dim=1)
```

```
correct += (predictions == labels).sum().item()
```

```
total += labels.size(0)
```

```
accuracy = 100 * correct / total
```

```
print(f"Total Accuracy: {accuracy:.2f}%")
```

```
torch.save(model.state_dict(), "minst_model.pth")
```

```
print("Model saved as minst_model.pth")
```

```
index = 0
```

```
image, true_label = test_data[index]
```

```
plt.show(image.squeeze(), cmap="gray")
```



```

plt.figure(figsize=(10, 10))
plt.title("Actual Label: {true_label}")
plt.axis("off")
plt.show()
image_flat = image.view(1, -1).to(device)
with torch.no_grad():
    output = model(image_flat)
    predicted_label = output.argmax(dim=1).item()
print("User Picked image index", index)
print("Actual Label", true_label)
print("Model Prediction", predicted_label)

```


● Code of CNN with Pytorch

import torch

import torch.nn as nn

image = torch.randn(1, 1, 28, 28)

conv = nn.Conv2d(

in_channels = 1,

out_channels = 8,

kernel_size = 3,

stride = 1,

padding = 1

)

output = conv(image)

print("Input image", image.shape)

print("Output Image", output.shape)

● Code of CNN with Pytorch Pooling

import torch

import torch.nn as nn

image = torch.randn(1, 8, 28, 28)

pool = nn.MaxPool2d(kernel_size = 2)

y = pool(x)

print("Before Pooling:", x.shape)

print("After Pooling:", y.shape)

• Code of conv Projects (MNIST)

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torchvision.datasets import MNIST
```

```
from torchvision import transforms
```

```
from torch.utils.data import DataLoader
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print("Using device:", device)
```

```
transform = transforms.ToTensor()
```

```
train_data = MNIST(
```

```
    root="data",
```

```
    train=True,
```

```
    download=True,
```

```
    transform=transform
```

```
)
```

```
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
```

```
test_data = MNIST(
```

```
    root="data",
```

```
    train=False,
```

```
    download=True,
```

```
    transform=transform
```

```
)
```

```
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)
```



```
class SimpleCNN(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.features = nn.Sequential(
```

```
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(2)
```

```
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(2)
```

```
        )
```

```
        self.classifier = nn.Sequential(
```

```
            nn.Linear(16 * 7 * 7, 64),
```

```
            nn.ReLU(),
```

```
            nn.Linear(64, 10)
```

```
        )
```

```
← def forward(self, x):
```

```
    x = self.features(x)
```

```
    x = x.view(x.size(0), -1)
```

```
    x = self.classifier(x)
```

```
    return x
```

```
model = SimpleCNN().to(device)
```

```
loss_fn = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
epochs = 3
```

```
for epoch in range(epochs):
```

```
    model.train()
```

```
    total_loss = 0
```


for images, labels in train-loader:

images = images.to(device)

labels = labels.to(device)

optimizer.zero_grad()

outputs = model(images)

loss = loss_fn(outputs, labels)

loss.backward()

optimizer.step()

total_loss += loss.item()

print(f"Epoch {epoch+1} / {epochs} | Loss:
{total_loss:.4f}")

model.eval()

correct = 0

total = 0

with torch.no_grad():

for images, labels in test_loader:

images = images.to(device)

labels = labels.to(device)

outputs = model(images)

predictions = outputs.argmax(dim=1)

correct += (predictions == labels).sum().
item()

total += labels.size(0)

accuracy = 100 * correct / total

print(f"Test accuracy: {accuracy:.2f}%")

torch.save(model.state_dict(), "simple_cnn_mnist.pth")

Print("Model saved")